CS-412 Software Security: Fuzzing Lab Report
Fuzzing of libpng 1.2.56

Sylvain Pichot Jeanne De Marmiés Erik Hübner Youcef Amar

Abstract

This report details a coverage-guided fuzzing campaign against libpng 1.2.56 using AFL++. We evaluate the library's security through both instrumented (white-box) and QEMU-mode (black-box) fuzzing. Our results quantify the performance impact of AddressSanitizer (ASan) and persistent mode, while investigating the code coverage achieved through mutation-based analysis.

1 Q1: HARNESS DESIGN

1.1 Entry Point Justification

For our fuzzing campaign, we selected the `libpng 1.2.56` decoder as our primary target. The entry point of our harness utilizes the low-level API sequence: `png_read_info()`, `png_set_expand()`, `png_read_update_info()`, and `png_read_row()`.

Justification: The decoder represents the most critical attack surface in any image processing library because it handles untrusted, complex bitstreams from external sources. We chose the low-level API over high-level convenience functions (like `png_read_png`) to gain granular control over the library's internal state. Specifically, by manually calling `png_set_expand()`, we force the fuzzer to explore transformation paths (e.g., converting indexed colors to RGB) that are often prone to memory corruption and were the source of historical vulnerabilities like CVE-2015-8126.

Alternatives Considered:

- **Encoder:** Rejected because encoders typically process trusted data already in memory. Fuzzing the encoder is less likely to yield security-relevant vulnerabilities in a real-world threat model.
- **High-level API (`png_read_png`):** Rejected because it automatically allocates memory for the entire image based on header dimensions. This makes the fuzzer highly susceptible to Out-of-Memory (OOM) "zip bombs," where a malformed header claims massive dimensions, crashing the fuzzer without discovering a real bug.
- **Other Libraries (Sixel/SDL):** While interesting, `libpng` 1.2.56 was selected because its documented historical bugs (e.g., CVE-2016-10087) provide a robust benchmark for validating our instrumentation and ASan setup.

1.2 Data Flow and Guard Rationale

The data flow originates from `AFL++`, which provides mutated inputs via a file path in `argv[1]`. This file is opened and linked to the `png_struct` via `png_init_io()`, serving as the source for all subsequent library calls.

Rationale for Code Guards:

- **`setjmp(png_jmpbuf(png))`**: This is the most critical guard for fuzzer stability. Since `libpng` uses `longjmp` for internal error handling (e.g., on malformed chunks), this guard catches those jumps. Without it, every invalid PNG would cause a process termination that AFL++ would incorrectly flag as a "crash," leading to thousands of false positives.
- **Dimension Limits (`height * rowbytes < 10MB`)**: This prevents OOM DoS attacks against the fuzzer. Mutated headers often contain gargantuan width/height values; this guard ensures we only allocate memory for reasonable sizes, maintaining high execution speed (exec/s).
- **`png_read_end()`**: This call is vital for reaching ancillary chunks (like `tEXt` or `zTXt`) located after the IDAT stream. This ensures coverage for metadata-related vulnerabilities such as CVE-2016-10087.
- **Strict Cleanup**: By calling `png_destroy_read_struct`, we ensure that memory is freed between every iteration. This is essential for long-running campaigns and persistent mode to prevent "memory accumulation" which would eventually cause a crash unrelated to the library's logic.

2 Q2: INSTRUMENTATION AND SANITIZERS

2.1 Compiler Flags and Instrumentation

We used the following flags during the white-box build phase:

- `CC=afl-clang-fast`: This selects the AFL++ LLVM-based compiler wrapper. It performs IR-level instrumentation by injecting coverage callbacks at every

branch during compilation. This allows AFL++ to track edge coverage in a 64KB shared memory bitmap.

- `-fsanitize=address`: Enables AddressSanitizer (ASan). ASan uses "shadow memory" (mapping 8 bytes of application memory to 1 byte of shadow memory) to track memory state. It places "redzones" around stack and heap objects to detect out-of-bounds accesses immediately.

- `-g`: Includes debug symbols. While not strictly necessary for fuzzing, it is vital for triage (Q5) to map crash addresses back to source code lines.

- `-O1`: We chose moderate optimization. High optimization (`-O3`) can sometimes inline vulnerable code or optimize away "useless" memory accesses that ASan needs to monitor.

2.2 Library Patches: CRC Removal

We applied the `libpng-nocrc.patch` provided in the AFL++ utilities. This patch modifies `png_crc_finish()` to return 0 (success) unconditionally.

Impact on Path Discovery: This is the most critical modification for `libpng`. Every PNG chunk contains a 4-byte CRC-32 checksum. When AFL++ mutates the chunk data (the "Havoc" stage), the checksum becomes invalid. Without this patch, the library calls `png_error()` and terminates immediately upon seeing an invalid CRC.

If omitted, the fuzzer would be trapped at the library's "front door." It would find thousands of inputs that trigger the same CRC error path but would never reach the deeper, vulnerable logic (decompression, palette handling, or pixel transformations). The patch essentially disables the data integrity gatekeeper, allowing mutated inputs to reach the complex attack surface.

2.3 Impact of ASan Omission

If `ASan` were omitted, the fuzzer would only detect crashes that cause a `SIGSEGV` (e.g., jumping to invalid memory). Subtle bugs like a 1-byte heap-buffer-overflow or an out-of-bounds read that does not hit unmapped memory would go completely unnoticed, resulting in a false sense of security.

3 Q3: SEED CORPUS AND DICTIONARY

Describe your seed selection strategy and dictionary entries. Quantify the contribution of the dictionary versus havoc/splice strategies using metrics from the AFL++ status screen.

4 Q4: CAMPAIGN ANALYSIS

The white-box campaign ran for 75,277 seconds and executed 62,454,465 test cases at 829.65 exec/s. Final AFL++ status metrics were: corpus count 1,103, stability 100.00%, bitmap coverage 31.69% (974 of 3,074 edges), cycles done 323, and 0 crashes / 0 hangs.

The edge-discovery curve in `plot_output/edges.png` shows a three-phase shape: an aggressive early discovery phase, a long diminishing-returns middle phase, and a final plateau. From `findings/default/plot_data`, coverage milestones were:

- 689 edges (22.41%) at 61 s — early rapid discovery
- 783 edges (25.47%) at 530 s — acceleration phase
- 935 edges (30.42%) at 7,183 s — approaching saturation
- 959 edges (31.20%) at 28,461 s — diminishing returns
- 973 edges (31.69%) at 49,478 s — final new edge
- 974 edges (31.69%) at 54,735 s — plateau reached

This timestamped progression supports a saturation argument. After roughly 15.2 hours, the campaign stopped finding new edges altogether, while the corpus still grew from 1,063 inputs at 54,735 s to 1,103 inputs at the end of the run. In other words, later mutations mostly produced new inputs that exercised already-known paths rather than discovering new control-flow. The campaign therefore reached practical saturation for this harness/library pair.

5 Q5: CRASH TRIAGE

Our white-box campaign found no real crashes after 62.4M executions. This is acceptable and defensible for `libpng 1.2.56`: it is a heavily analyzed target, and the CRC patch plus harness guards significantly reduced false positives and OOM-style noise.

We validated our setup by injecting a synthetic bug. A single heap-buffer-overflow was added inside `png_read_row()` immediately after the library copies the decompressed row into `prev_row`. This is applied only to a separate copy of the library (`libpng-1.2.56_bugged`), while the original `libpng-1.2.56` remains untouched for the normal campaign. The injected code writes past the `prev_row` allocation, producing a deterministic out-of-bounds write that ASan catches when the harness is compiled with `-fsanitize=address`.

Why this location: `png_read_row()` is on the normal input-processing path and is exercised by the existing harness loop.

Reproduction steps (exact commands):

```
# From host: enter the container
make run-docker

# Inside container: build and fuzz the bugged
    harness
make fuzz-bugged

# Reproduce a crash directly
./harness_bugged findings-bugged/default/
    crashes/id:000000*

# Minimize the crash input
afl-tmin -i findings-bugged/default/crashes/
    id:000000* \
         -o minimized.png -- ./harness_bugged
             @@

# Get full ASan stack trace
ASAN_OPTIONS=symbolize=1 ./harness_bugged
    minimized.png
```

AFL++ found 10 crashes within 85 seconds. The minimized input was reduced from 357 to 325 bytes by `afl-tmin`. The resulting ASan stack trace (see Figure 5 and Sub-Appendix C) confirms a heap-buffer-overflow WRITE of 1 byte at `pngread.c:789`, 4 bytes before an 885-byte `row_ptr` allocation.

This synthetic bug demonstration proves that (1) our build pipeline includes AddressSanitizer, (2) ASan is active at runtime and detects memory errors, and (3) AFL++ can find crashes in instrumented binaries within minutes. Since the 62.4M-execution white-box campaign produced no crashes despite reaching 31.69% coverage, the most reasonable conclusion is that `libpng 1.2.56` does not contain an easily reachable memory-corruption bug under our harness configuration.

6 Q6: ATTACK SURFACE ANALYSIS

6.1 Applications and Attack Scenarios

`libpng` is critical for cross-platform image rendering. We identify two primary attack vectors:

- **Mozilla Firefox (Web Rendering):** Browsers automatically parse PNGs from untrusted web servers. A malicious PNG triggering a buffer overflow (e.g., CVE-2015-8126) allows **code execution within the renderer process**, which an attacker could chain with a sandbox escape to achieve full Remote Code Execution (RCE).

- **ImageMagick (Server-side Processing):** Web backends use this to process user uploads. A malformed chunk triggering a NULL-pointer dereference (CVE-2016-10087) causes a **Denial of Service (DoS)** or memory disclosure, crashing the thumbnail generation service.

6.2 Coverage Gaps and Security Implications

Our harness focuses on linear decoding, leaving several critical paths unexercised:

1. **Encoding API (`png_write_*`):** Functions in `pngwrite.c` and `pngwtran.c` are never called. Although encoders typically handle trusted data, vulnerabilities can arise during raw-to-chunk conversion if metadata checks are inconsistent.

2. **Adam7 Interlacing (`png_do_read_interlace`):** Our harness reads rows linearly and does not invoke interlacing logic in `pngrutil.c`. Interlacing involves complex, non-linear coordinate math that is historically prone to **off-by-one errors** and heap corruption when calculating buffer offsets for incomplete passes.

An attacker targeting Firefox's image rendering pipeline or ImageMagick's upload processing could exploit either gap without our campaign detecting it.

7 Q7: BINARY-ONLY FUZZING WITH QEMU MODE

7.1 Comparative Analysis

We executed a parallel campaign using AFL++ QEMU mode (`-Q`) against an uninstrumented `libpng` binary compiled with standard `gcc` and no sanitizers. The results after ∼35 minutes (Figure 3) are summarized in Table 1:

Metric	Instrumented	QEMU Mode (-Q)
Exec Speed	861.8 exec/s	2,384 exec/s
Map Density	8.65%	0.74%
Corpus Count	614 items	872 items

Table 1: Comparison metrics after 35 minutes of fuzzing.

7.2 Analysis and Differences

- **Execution Speed:** QEMU mode achieved 2,384 exec/s vs. 861.8 exec/s for the instrumented build. The instrumented build is slowed by **AddressSanitizer (ASan)**, which inserts a shadow-memory check before every memory access. Without ASan, native compile-time instrumentation is typically 10–50× faster than QEMU emulation; here ASan inverts the relationship.

- **Map Density (Coverage):** The instrumented campaign reached 8.65% edge coverage vs. only 0.74% for QEMU. Compile-time instrumentation captures every branch transition at the IR level, whereas QEMU's block-level translation logs coarser basic-block transitions and frequently collapses multiple branches into a single translated block.

- **Corpus Count:** QEMU produced a larger corpus (872 vs. 614 items) despite lower coverage. Without precise edge feedback, AFL++ accepts test cases that do not exercise genuinely new control-flow, resulting in a larger but more redundant corpus.

7.3 Coverage Collection Mechanism

Compile-Time Instrumentation (`afl-clang-fast`) statically injects coverage checkpoints at every branch during compilation, adding only a few instructions per edge with no translation overhead at runtime.

QEMU Mode uses **Dynamic Binary Translation** via the Tiny Code Generator (TCG): guest instruction blocks are translated to host code at runtime, and AFL++ hooks the TCG pipeline to log basic-block transitions. This emulation layer adds substantial per-block overhead and produces coarser coverage maps, explaining the 11.7× lower map density despite a higher raw execution rate.

8 Q8: INSTRUMENTATION DEPTH & PERFORMANCE

8.1 Instrumented Edge Counts

At startup, `afl-fuzz` reports the total number of instrumented edges in the harness binary via `fuzzer_stats`. For our final `harness_whitebox` binary, the total is **3,074 edges**. This count covers both the libpng 1.2.56 library code and the small `main()` function in our harness (`src/harness.c`). The harness itself contributes only a handful of edges (file open, argument check, cleanup), so the overwhelming majority of the 3,074 edges originate from libpng's parsing, decompression, and transformation pipeline.

After 75,277 seconds of fuzzing, the campaign reached **974 edges (31.69% map density)**. The remaining 2,100 edges (68.31%) were never triggered. Many code paths in libpng require very specific input conditions to activate, such as Adam7 interlacing (`png_do_read_interlace` in `pngrutil.c`), 16-bit depth processing, palette-mode expansion, or rare ancillary chunks (`zTXt`, `iTXt`). Mutation-based fuzzing without structure-aware generation struggles to produce the precise byte sequences needed to unlock these paths, causing the campaign to plateau at roughly one third of total coverage.

8.2 Execution Speed Benchmark

We measured `exec speed` under three configurations using the same libpng 1.2.56 target compiled with the CRC patch applied (see Figure 4 for config (1), Figure 1 for config (2), and Figure 6 for config (3)):

No sanitizer + fork (6,607 exec/s): Without ASan, there is no shadow memory to initialize or maintain, and no redzone checks on every allocation and access. The process still pays the `fork()` cost per test case, which caps throughput.

Configuration	Exec Speed (exec/s)
(1) No sanitizer + fork mode	6,607
(2) ASan + fork mode	830
(3) ASan + persistent mode	44,400

Table 2: Execution speed across three instrumentation configurations.

ASan + fork (830 exec/s): Adding ASan reduces throughput by roughly **8**×. ASan maps one shadow byte for every 8 bytes of application memory and inserts inline checks before every memory access. For libpng, which performs many small heap allocations per image (row buffers, info structs, zlib state), this overhead accumulates significantly per execution.

ASan + persistent mode (44,400 exec/s): Persistent mode eliminates the `fork()` cost entirely by looping inside a single process using the `__AFL_LOOP()` macro. The fuzzer delivers test cases via shared memory, yielding a **53× speedup** over ASan + fork. ASan remains active but its initialization cost is amortized over 10,000 iterations before the process restarts.

ACKNOWLEDGMENTS

Optional section for acknowledging collaborators or tools.

AVAILABILITY

Specify where the fuzzing environment (Dockerfile, Makefile, Harness) and campaign artifacts can be found.

9 CONCLUSION

Final summary of campaign efficacy.

Appendix

A SUB-APPENDIX A: AFL++ STATUS SCREENS

Contains screenshots of the AFL++ status UI for both campaigns.

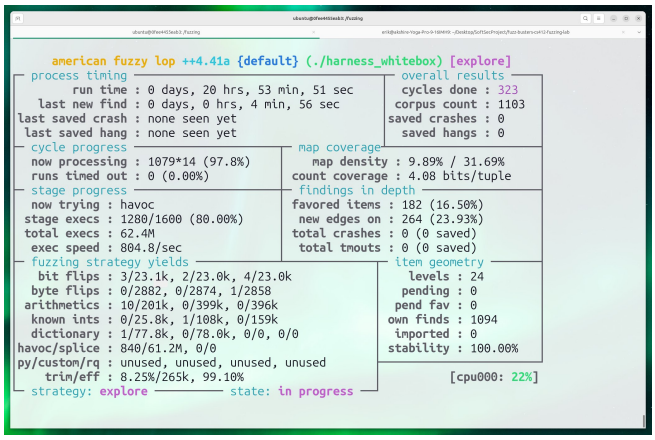

Figure 1: AFL++ UI for the instrumented campaign.

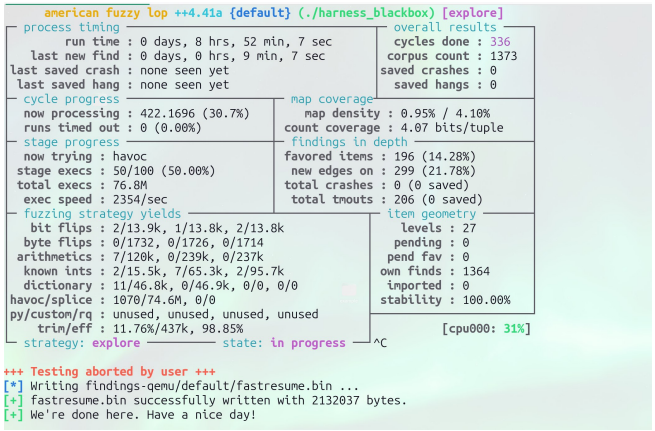

Figure 2: AFL++ UI for the binary-only (QEMU) campaign.

B SUB-APPENDIX B: COVERAGE PLOTS

Contains the generated plots showing coverage (edges) over time.

C SUB-APPENDIX C: TRIAGE AND SOURCE CODE

Contains the harness source code and crash triage reports.

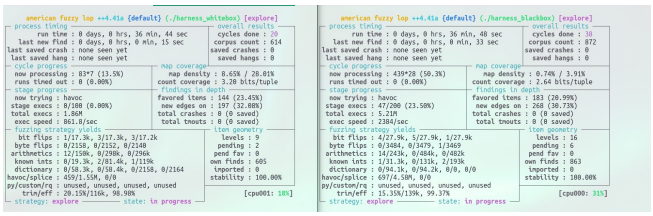

Figure 3: Comparison of AFL++ status screens after 35 minutes: Instrumented White-box (Left) vs. QEMU Black-box (Right).

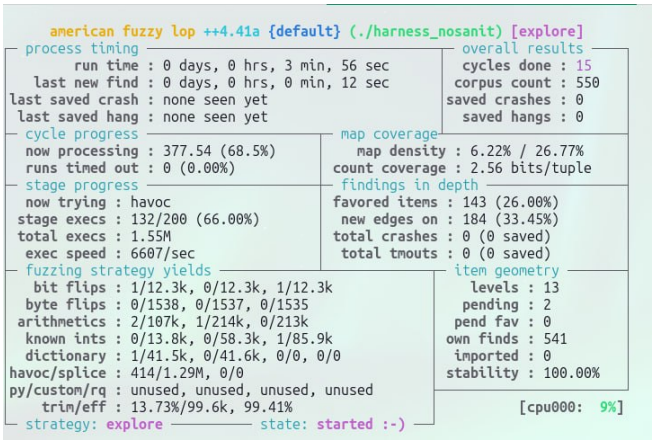

Figure 4: AFL++ status screen for the campaign without AddressSanitizer.

```
==1966515==ERROR: AddressSanitizer: heap-
    buffer-overflow on address
0x51800000047c at pc 0x5c1dd933c6cb bp 0
    x7ffdb2a6a490
WRITE of size 1 at 0x51800000047c thread T0
    #0 0x5c1dd933c6ca in png_read_row
          /fuzzing/libpng-1.2.56_bugged/
              pngread.c:789:48
    #1 0x5c1dd932fbbb in main /fuzzing/src/
        harness.c:49:17

0x51800000047c is located 4 bytes before 885-
    byte region
[0x518000000480,0x5180000007f5)
allocated by thread T0 here:
    #0 0x5c1dd92ee2f3 in malloc (/fuzzing/
        harness_bugged+0xd12f3)
    #1 0x5c1dd932fb73 in main /fuzzing/src/
        harness.c:46:29

SUMMARY: AddressSanitizer: heap-buffer-
    overflow
/fuzzing/libpng-1.2.56_bugged/pngread.c
    :789:48 in png_read_row
==1966515==ABORTING
```

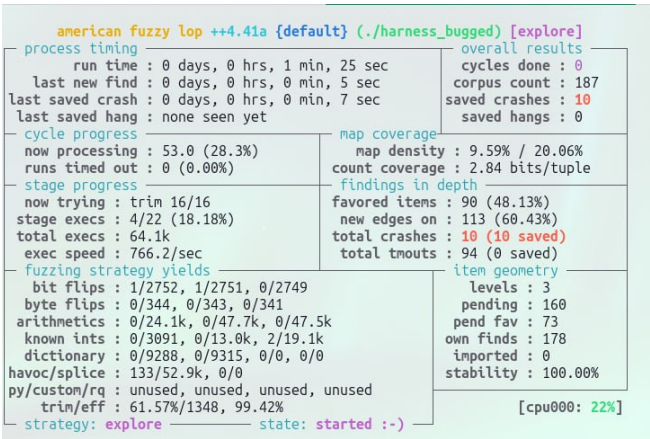

Figure 5: AFL++ status screen showing the detection of unique crashes when fuzzing the bugged version of the harness.

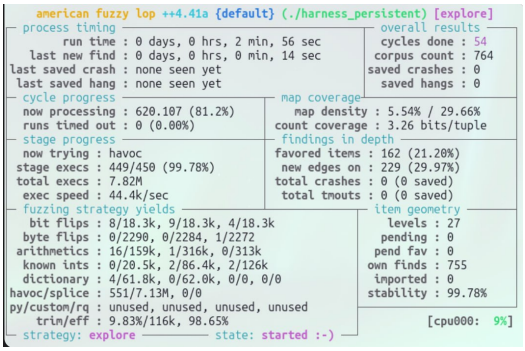

Figure 6: Maximum performance: Native build using Persistent Mode.

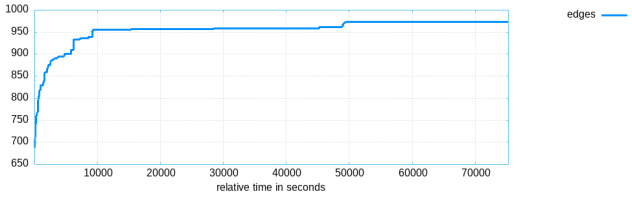

Figure 7: Edge discovery plot (Instrumented).

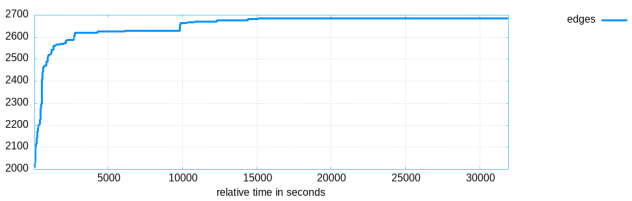

Figure 8: Edge discovery plot (QEMU mode).

Listing 1: ASan Stack Trace for Q5 Triage